

C vs Assembly

Date

No.

C language → C is a high level, imperative programming language designed for efficient system programming. It offers abstractions like variables, data types, functions, and control structures (loops, conditionals), allowing programmers to express logic in a human-readable & portable manner. C code focuses on the overall program flow and data manipulation without requiring direct interaction with the underlying hardware registers or memory addresses.

Assembly language → is a low level programming language that directly corresponds to the machine code instructions of a specific processor architecture. It uses mnemonic codes to represent operations (e.g. MOV for move, ADD for add) and directly manipulates CPU registers, memory locations, and I/O ports. Assembly code is highly machine dependent and requires a deep understanding of the target architecture.

Comparison through gcc -S: The gcc -S command compiles a C source file into an assembly language file (.s extension) without performing the assembly/linking steps. This process reveals how the high-level constructs of C are translated into low-level assembly instructions.

Key differences: Abstraction level: C provides a high level of abstraction, abstracting away hardware details. Assembly language operates at the lowest level, directly manipulating hardware resources.

Readability & Maintainability: C code is generally more readable, understandable, and maintainable due to its structured nature and use of meaningful identifiers.

Assembly code is significantly less readable and more challenging to maintain, requiring detailed knowledge of the target architecture's instruction set.

Portability: C code is designed to be highly portable across different architectures with minimal modifications. Assembly code is inherently non-portable, as it is tied to a specific processor's instruction set.

Performance & Control: Assembly language offers the potential for fine-grained control over hardware and highly optimized performance, as programmers can meticulously craft instructions for specific tasks. However, achieving this level of optimization requires significant expertise. C compilers, especially modern ones, are highly optimized and can often generate assembly code that rivals or even surpasses hand-written assembly in performance for general-purpose tasks.

Development Time: Developing applications in C is typically much faster and less error-prone than in assembly due to the higher level of abstraction and available libraries.

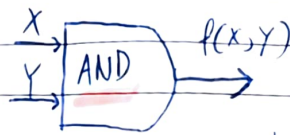
Conclusion: While C provides a powerful and portable platform for general-purpose programming, assembly language remains crucial for tasks requiring absolute control over hardware, extreme performance optimization in specific critical sections, or when interacting with unique hardware features not easily accessible through C. The gcc-`-S` command serves as a valuable tool for understanding the bridge between these two levels of programming, illustrating how high-level C constructs are ultimately realized through low-level machine instructions.

Digital Fundamentals - Logic Gates

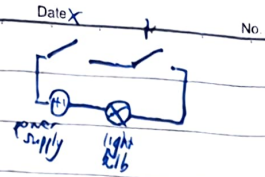
Boolean Functions

X AND Y

X	Y	f
0	0	0
0	1	0
1	0	0
1	1	1

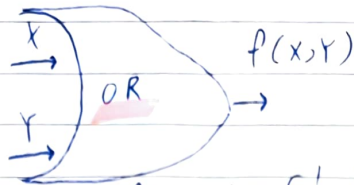


$$f(x,y) = \begin{cases} 1 & \text{when } x=1 \text{ \& } y=1 \\ 0 & \text{otherwise} \end{cases}$$



X OR Y

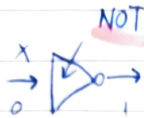
X	Y	OR
0	0	0
0	1	1
1	0	1
1	1	1



$$f(x,y) = \begin{cases} 1 & \text{when } x=1 \text{ OR } y=1 \\ 0 & \text{otherwise} \end{cases}$$

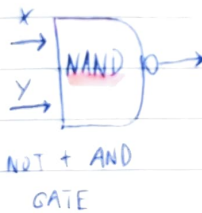


X	Not	Inverts
0	1	
1	0	

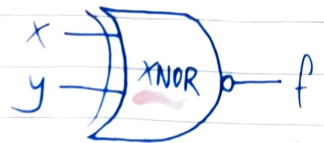


X	Y	NAND
0	0	1
0	1	1
1	0	1
1	1	0

Inverts AND Gate



NOT + AND
GATE



X	Y	f
0	0	1
0	1	0
1	0	0
1	1	1

[2 1s or 0s]

X	Xor	Y
0	0	0
0	1	1
1	0	1
1	1	0

1 if different inputs



X	Y	f
0	0	1
0	1	0
1	0	0
1	1	0

Digital Fundamentals - Conversion

Date

No

Decimal $\rightarrow$ Hexa

$$109_{10} \rightarrow \begin{array}{l} 109 \div 16 = 6 \text{ R } 13 \\ 6 \div 16 = 0 \text{ R } 6 \end{array} \uparrow 6D_{16}$$

$$219_{10} \rightarrow \begin{array}{l} 219 \div 16 = 13 \text{ R } 11 \\ 13 \div 16 = 0 \text{ R } 13 \end{array} \uparrow DB_{16}$$

$$853_{10} \rightarrow \begin{array}{l} 853 \div 16 = 53 \text{ R } 5 \\ 53 \div 16 = 3 \text{ R } 5 \\ 3 \div 16 = 0 \text{ R } 3 \end{array} \uparrow 355_{16}$$

Hexa	Decimal
A	10
B	11
C	12
D	13
E	14
F	15

Hexa $\rightarrow$ Decimal

$$32A_{16} \rightarrow 10 + 32 + 768 = 810_{10}$$

$$\begin{array}{c} \downarrow \\ \downarrow \times 16^0 \\ \downarrow \times 16^1 \end{array}$$

$$\times 16^2$$

$$B3F_{16} \rightarrow 15 + 48 + 2816 = 2879_{10}$$

$$\begin{array}{c} \downarrow \\ \downarrow \times 16^0 \\ \downarrow \times 16^1 \end{array}$$

$$\times 16^2$$

$$C3D9_{16} \rightarrow 9 + 208 + 768 + 49152 = 50137_{10}$$

$$\begin{array}{c} \downarrow \downarrow \downarrow \\ \downarrow \times 16^2 \times 16^1 \times 16^0 \end{array}$$

$$\times 16^3$$

Hexa $\rightarrow$ Dec

$$\begin{array}{l} \nearrow 0.75 \quad \nearrow 0.015625 \\ 12 \times 16^{-1} \quad 4 \times 16^{-2} \\ \uparrow \quad \nearrow \end{array}$$

$$28B.C4_{16} = 11 + 128 + 512 + 0.75 + 0.015625 \\ = 691.765625_{10}$$

$$\begin{array}{l} \downarrow \\ 11 \times 16^0 = 11 \\ \downarrow \\ 8 \times 16^1 = 128 \\ \downarrow \end{array}$$

$$2 \times 16^2 = 512$$

Binary $\rightarrow$ Decimal

$$10110 \rightarrow 16 + 4 + 2 = 22$$

$$16 \ 8 \ 4 \ 2 \ 1$$

$$1011 \rightarrow 1 + 2 + 8 = 11$$

$$8 \ 4 \ 2 \ 1$$

$$11011 \rightarrow 1 + 2 + 8 + 16 = 27$$

$$16 \ 8 \ 4 \ 2 \ 1$$

Decimal $\rightarrow$ Binary

$$37 \rightarrow \begin{array}{r} 1 \ 0 \ 0 \ 1 \ 0 \ 1 \\ \hline 32 \ 16 \ 8 \ 4 \ 2 \ 1 \end{array}$$

$$26 \rightarrow \begin{array}{r} 1 \ 1 \ 0 \ 1 \ 0 \\ \hline 16 \ 8 \ 4 \ 2 \ 1 \end{array}$$

$$14 \rightarrow \begin{array}{r} 1 \ 1 \ 1 \ 0 \\ \hline 8 \ 4 \ 2 \ 1 \end{array}$$

$$35 \rightarrow \begin{array}{r} 1 \ 0 \ 0 \ 0 \ 1 \ 1 \\ \hline 32 \ 16 \ 8 \ 4 \ 2 \ 1 \end{array}$$



OSTEP Chapter 1 - Introduction to operating systems

Goal of an OS: provide a virtualized, safe, and convenient environment for running programs

Main roles of an OS:

provide a virtualized, safe and convenient environment for running programs.

Main roles of an OS

1. Abstraction

- Hides hardware complexity from programs
- provides clean interfaces (files, process, virtual memory)

2. Resource Management

- CPU Scheduling
- Memory allocation
- I/O control
- Fairness + efficiency

3. Control / protection

- Isolates processes
- prevents interference
- Enforces permissions

Key Hw: Virtualization

- OS makes one physical machine look like many independent logical machines

Example:

Virtual CPU → process abstraction

Virtual memory → address space

Virtual disks → file system

Chapter 2 → Processes (The Abstraction)

This chapter defines what a "process" means in OSTEP, why it's important and how OS virtualizes the CPU.

- A process is a running program. The static program on disk becomes a "process" once loaded & executed.
- A process's state comprises:
 - It's address space (memory: code, stack, heap, static data)
 - CPU-related state: registers, program counter (PC), stack pointer — everything needed to resume execution later
 - OS-tracked metadata: open files / IO state, process ID (PID), scheduling information, etc
- Because there are limited physical CPUs, the OS uses CPU virtualization / time-sharing, switching between processes so that it "appears" multiple processes run simultaneously.
- Processes can be in multiple states: commonly Running, Ready (Ready to run, waiting to be scheduled), or Blocked (waiting for I/O or other event)

This abstraction is powerful: It lets users run many programs concurrently — even hundreds — even with one (or few) CPUs underlying.

Chapter 3 → Process API (creation, control, lifecycle)

This chapter discusses how an OS allows programs to create & manage processes — what interfaces the OS gives to use — programs

- In UNIX-style systems, typical system calls for processes include:
 - `fork()` — creates a new process (child) as a (nearly) identical copy of the parent
 - `exec()` — replace the process memory image with a new program (after `fork`, often) so that the child can start executing a different program
 - `wait()` — parent waits for child processes to finish; helps manage child lifetime & prevent zombie process

- `exit()` - terminate a process. OS cleans up process resources; parent may check exit status
- The book shows a typical workflow: a program calls `fork()`, child does `exec()` to load a new program, parent does `wait()`
- It also touches upon the "process control block" (PCB) - the internal data structure OS uses to track all necessary info about each process (state, registers, memory, open files, etc.) so it can suspend/resume processes (ie context switching.)

